

CompoScript Programming Language Documentation

طراحی کامپایلر و زبان‌های برنامه‌نویسی

دکتر فاطمه قاسمی

پاییز ۱۴۰۴

فهرست مطالب

۱) مقدمه 2
۲) فلسفه طراحی سیستمی 2
۳) ساختار کلی 3
۳-۱) Component-ها، پورت‌ها، سیگنال‌ها و نمونه‌ها 3
۳-۲) ایجاد نمونه از Component ها 5
۴) مدل سازه‌ای (Component Model) 6
۴-۱) تعریف componenttype ... endcomponent 6
۴-۲) پورت‌ها و جهت‌ها: in/out data 7
۴-۳) سیگنال و متغیرها 7
۴-۴) نمونه‌سازی و اتصال کامپوننت‌ها 8
۴-۵) پارامترها و مقادیر ثابت (const) 8
۵) مدل زمان کل سیستم (Global Time) 9
۵-۱) اجرای هم‌زمان عبارات 9
۵-۲) زمان کل سیستم و progress time 10
۵-۳) مفهوم timer و تیک‌های زمانی 10
۵-۴) تکرار تناوبی (repeat) 11
۶) تولید ساختاری 11
۶-۱) سازه‌ی expand، index، do...enddo 11
۶-۲) الگوهای تکرار ساختاری و نام‌گذاری بلوک‌ها 12
۷) انواع داده 13
۷-۱) انواع داده‌های عددی (Numeric Data Types) 13
۷-۲) انواع داده‌های رشته‌ای (String Data Types) 14
۷-۳) انواع داده‌های آرایه‌ای (Array Types) 15
۷-۴) داده‌های متنوع 15
۷-۵) متغیرهای تولیدی (Generative Variables) 17
۸) عملگرها 18
۸-۱) عملگرهای حسابی 18
۸-۲) عملگرهای مقایسه‌ای 19
۸-۳) عملگرهای منطقی 20
۸-۵) عملگر تخصیص 20
۸-۶) اولویت عملگرها 21
۸-۷) عملگر شرطی 22
۹) توابع پیش‌فرض 22

۱) مقدمه

در حوزه طراحی و تحلیل سیستم‌های پیچیده، زبان‌های مدل‌سازی ابزار اصلی برای توصیف رفتار و تعامل اجزای سیستم هستند. این زبان‌ها امکان می‌دهند طراحان و محققان، ساختار و رفتار سیستم‌ها را در سطحی انتزاعی مدل کرده و پیش از پیاده‌سازی، عملکرد آن‌ها را بررسی و تحلیل کنند. نمونه‌هایی از چنین زبان‌هایی شامل mCRL2 هستند که برای مدل‌سازی سیستم‌های توزیع‌شده و همزمان طراحی شده‌اند.

هدف این پروژه طراحی زبانی به نام CompoScript است. این زبان به گونه‌ای طراحی شده است که شما بتوانید با سینتکسی ساده و قابل فهم، فرایندهای همروند را مدل‌سازی کرده و کامپوننت‌های مختلف سیستمی را به یکدیگر متصل کنید، بدون آنکه پیچیدگی‌های اضافی زبان‌های مدل‌سازی کامل شما را گیج کند. همچنین اجرای کد در CompoScript به صورت یکجا انجام می‌شود و نه خط به خط، که جزئیات آن در بخش بعدی به‌طور کامل توضیح داده شده است.

افزون بر این، CompoScript از سیستم استنتاج نوع Type Inference بهره می‌برد؛ بدین معنا که نیازی به تعریف صریح نوع داده‌ها نیست و زبان ما به‌طور خودکار نوع مقادیر و متغیرها را از روی Context و نحوه استفاده آن‌ها استنتاج می‌کند. این ویژگی موجب می‌شود کدنویسی روان‌تر و کوتاه‌تر شود در حالی که همچنان از مزایای بررسی نوع در زمان تحلیل مدل بهره‌مند می‌شویم.

۲) فلسفه طراحی سیستمی

زبان‌های طراحی سیستمی (System Design Languages) برای مدل‌سازی رفتار و ساختار سیستم‌های پیچیده، همزمان و کامپوننت‌محور توسعه یافته‌اند. این زبان‌ها امکان می‌دهند طراحی‌ها به صورت ماژولار و پارامتریک بیان شوند و قبل از پیاده‌سازی فیزیکی، عملکرد و تعامل اجزا بررسی شود.

CompoScript تمرکز آن بر سادگی و وضوح سینتکس است. فلسفه‌ی طراحی این زبان بر چند اصل کلیدی استوار است:

- ماژولار بودن: کامپوننت‌ها (components) به صورت مستقل و قابل نمونه‌سازی طراحی شده‌اند تا بازاستفاده و ترکیب آن‌ها آسان باشد.

- همگامی با رفتار سیستم‌ها: تمام کامپوننت‌ها در CompoScript به‌صورت همزمان اجرا می‌شوند و نه خط به خط، که امکان مدل‌سازی تعاملات همزمان و ترتیب رویدادها را فراهم می‌کند.

- مدل‌سازی با زمان کلی سیستم: در CompoScript، زمان کل سیستم در ابتدای اجرای برنامه صفر است و یک scheduler داخلی ترتیب و زمان‌بندی تمام رویدادها سیستم را مدیریت می‌کند، به‌گونه‌ای که هر فرایند در زمان مناسب اجرا می‌شود.

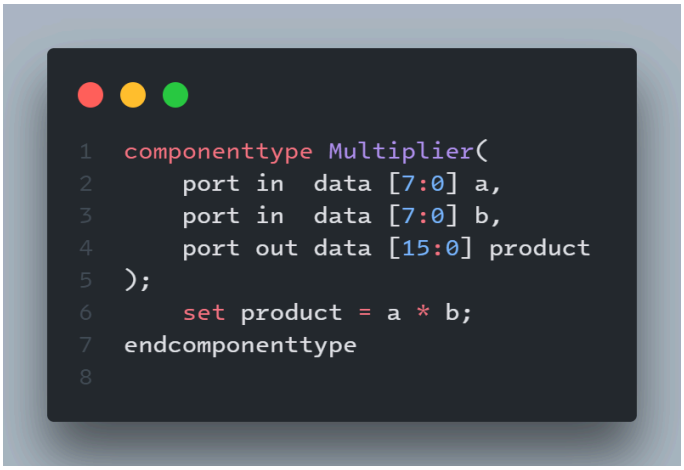
به این ترتیب، فلسفه طراحی CompoScript ترکیبی از سادگی، ماژولار بودن و قابلیت تولید کد پارامتریک است که امکان طراحی سیستم‌های هم‌روند واقعی و مقیاس‌پذیر را فراهم می‌کند.

۳) ساختار کلی

زبان CompoScript از ساختار ماژولار و واضحی برای تعریف کامپوننت‌ها، اتصالات و سناریوهای تست برخوردار است. در ادامه به جزئیات ساختار کلی زبان و قواعد نحو پرداخته شده است:

۳-۱) Component-ها، پورت‌ها، سیگنال‌ها و نمونه‌ها

- یک Component با کلمه‌ی کلیدی componenttype شروع شده و با endcomponenttype خاتمه می‌یابد. به مثال زیر توجه کنید. یک Component با نام Multiplier تعریف و ورودی‌ها، خروجی و بدنه آن نوشته شده است.



```
componenttype Multiplier(
    port in  data [7:0] a,
    port in  data [7:0] b,
    port out data [15:0] product
);
    set product = a * b;
endcomponenttype
```

- پورت‌ها با نوع in data یا out data تعریف می‌شوند و می‌توانند به صورت یک مقدار تکی یا یک بردار چندعنصری ([N-1:0]) تعریف شوند.

کامنت‌ها: با // شروع می‌شوند و تا انتهای خط ادامه دارند.کامنت‌ها تاثیری روی اجرای برنامه ندارند و صرفا برای توضیح کد به کار می‌روند.

Component ها: باید نام معنادار و بدون فاصله باشند، مانند Top_Basic یا CounterN.

پورت‌ها و سیگنال‌ها: حروف کوچک و با _ برای جدا کردن کلمات معمول است، مانند timer_in, q0.

نمونه‌ها (Instances): نام نمونه‌ها می‌تواند هر شناسه‌ی معتبری باشد (مثلا u_inc, b0). با این حال، برای افزایش خوانایی و تشخیص سریع‌تر نقش هر نمونه در کد، معمولا از پیشوندهایی مانند u_ (مخفف unit) یا b (برای بلوک‌ها) استفاده می‌شود.

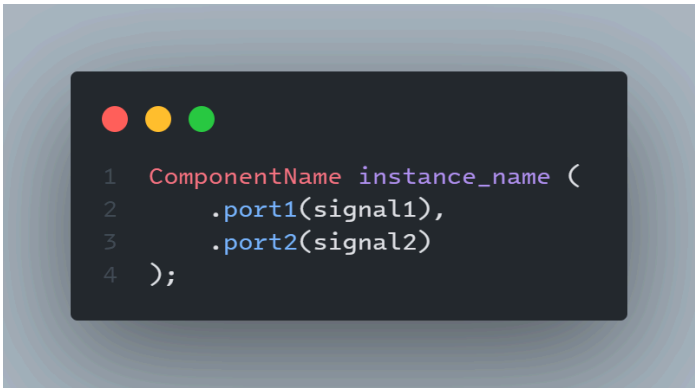
کلیدواژه‌ها: هیچ شناسه‌ای نباید معادل کلیدواژه‌های CompoScript باشد. جدول کلیدواژه‌ها به شرح زیر است:

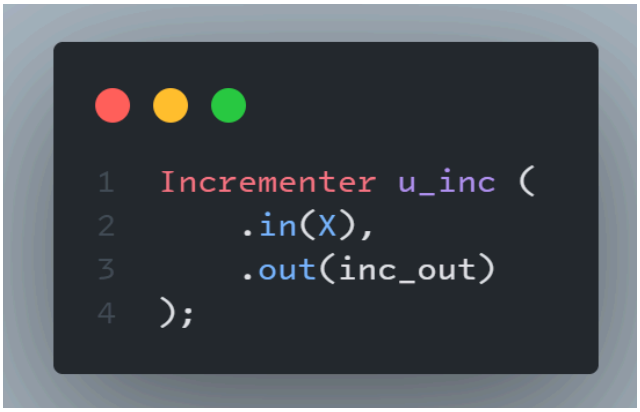
componenttype	data	expand	repeat	period	FOREVER
endcomponenttype	set	endexpand	display	for	port
in	parameter	do	if	configuration	

out	index	enddo	else	endconfiguration	

۳-۲) ایجاد نمونه از Component ها

نمونه‌سازی یک Component با نوشتن نام نمونه و اتصال پورت‌ها انجام می‌شود.





برای Component های پارامتریک، پارامترها به صورت ((.PARAM(value)# داده می‌شوند:



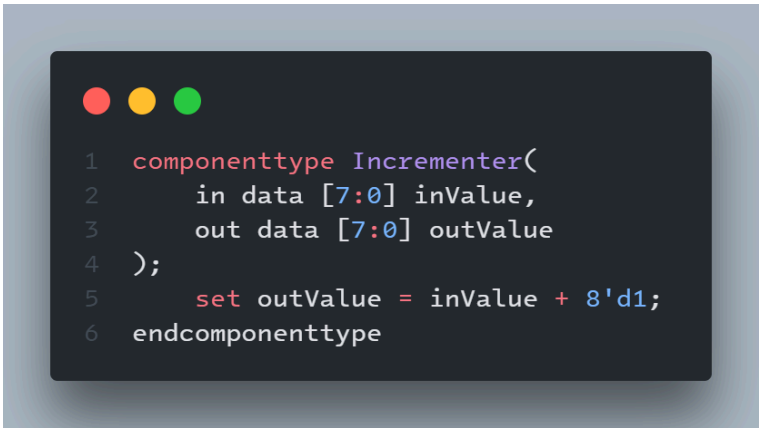
```
AdderN #(.N(8)) u_add8 (
    .X(A),
    .Y(B),
    .Cin(1'b0),
    .Sum(sum8),
    .Cout(cout)
);
```

ترکیب expand و index می‌تواند نمونه‌های تکراری را خودکار ایجاد کند و نیاز به نوشتن کد طولانی را از بین می‌برد.

۴) مدل سازه‌ای (Component Model)

۴-۱) تعریف componenttype … endcomponent

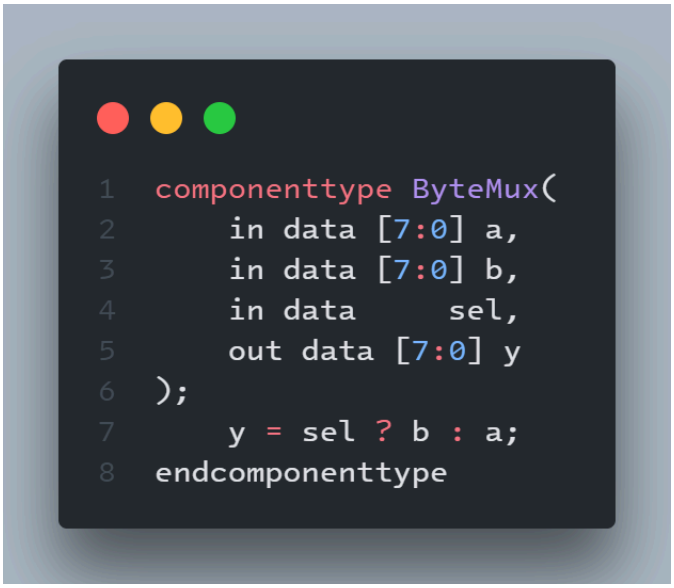
یک Component واحد سازه‌ای اصلی در CompoScript است. هر مولفه با کلیدواژه‌ی component آغاز و با endcomponent خاتمه می‌یابد. بدنه‌ی مولفه شامل تعریف پورت‌ها، داده‌های داخلی، پارامترها/ثابت‌ها و اتصالات است. دقت کنید هر مولفه باید نام یکتا داشته باشد و بدنه می‌تواند شامل تعریف زیرمولفه‌ها (نمونه‌سازی از Components دیگر) باشد:



```
componenttype Incrementer(
    in data [7:0] inValue,
    out data [7:0] outValue
);
    set outValue = inValue + 8'd1;
endcomponenttype
```

۴-۲) پورت‌ها و جهت‌ها: in/out data

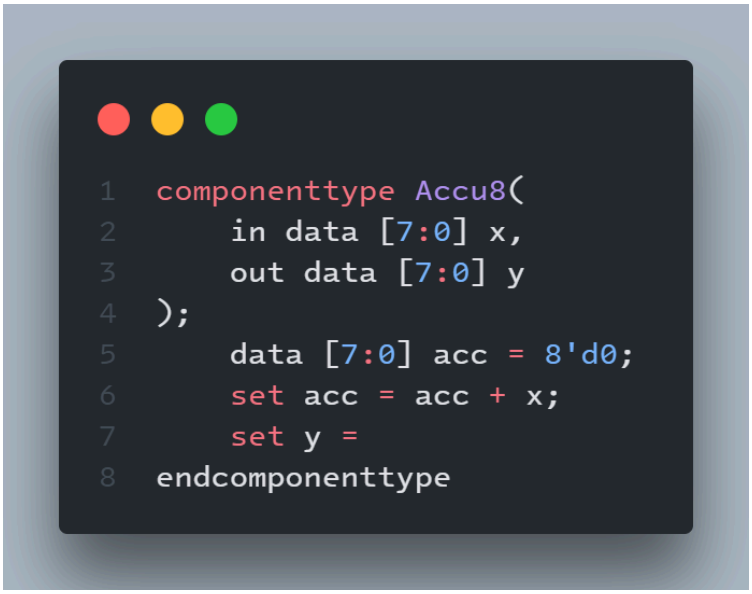
پورت‌ها با نوع data و جهت‌های in/out تعریف می‌شوند که می‌توانند به صورت آرایه هم باشند:



```
componenttype ByteMux(
    in data [7:0] a,
    in data [7:0] b,
    in data     sel,
    out data [7:0] y
);
    y = sel ? b : a;
endcomponenttype
```

۴-۳) سیگنال و متغیرها

در داخل یک کامپوننت می‌توان مقدار پورت خروجی/متغیر داخلی را مشخص کرد:



```
componenttype Accu8(
    in data [7:0] x,
    out data [7:0] y
);
    data [7:0] acc = 8'd0;
    set acc = acc + x;
    set y =
endcomponenttype
```

۴-۴) نمونه‌سازی و اتصال کامپوننت‌ها

اتصال زیرکامپوننت‌ها شبیه Verilog و به‌صورت Named port connection انجام می‌شود:

```
componenttype Top(
    in data [7:0] X, Y,
    out data [15:0] result
);

    data [7:0] inc_out;

    Incrementer u_inc (
      .in(X),
      .out(inc_out)
    );

    Multiplier u_mul (
      .a(inc_out),
      .b(Y),
      .product(result)
    );
endcomponenttype
```

۴-۵) پارامترها و مقادیر ثابت (const)

برای پیکربندی کامپوننت‌ها می‌توانید از پارامترهای ثابت استفاده کنید:

```
component FullAdder(
   in data A, B, Cin,
   out data Sum, Cout
);
   set Sum = A ^ B ^ Cin;
   set Cout = (A & B) | (B & Cin) | (A & Cin);
endcomponent

component AdderN #(
   const int N = 8
)(
   in data [N-1:0] X, Y,
   in data       Cin,
   out data [N-1:0] Sum,
   out data       Cout
);
endcomponent
```

const مقدار پیش‌فرض را تعیین می‌کند که با استفاده از #(.N (...)) یک نمونه از آن ایجاد می‌شود:

```
AdderN #(.N(8)) u_add8 ( .X(A), .Y(B), .Cin(1'b0), .Sum(sum8), .Cout(cout) );
```

۵) مدل زمان کل سیستم (Global Time)

در زبان CompoScript مانند اکثر زبان‌های توصیف سیستم، برنامه طبق یک مدل زمانی پیش می‌رود که تقدم و تاخر رویدادهای تعریف‌شده در برنامه و زمان‌بندی اجرای آن‌ها را مشخص می‌کند.

۵-۱) اجرای هم‌زمان عبارات

در زبان CompoScript، مولفه‌های مختلف برنامه به صورت موازی و هم‌زمان اجرا شده و در این زبان، اجرا به صورت ترتیبی (Sequential) وجود ندارد. برای مثال، در کد زیر، اجرای دو قطعه کد expand و

set به صورت موازی و هم‌زمان صورت می‌گیرد و توجه به این مورد در هنگام پیاده‌سازی کامپایلر مربوطه حائز اهمیت است. (انتساب‌ها با دستور set انجام می‌شوند)

```
index i;
expand
   for (i = 0; i < N; i++) do : G
      FDRE #(.INIT(1'b0)) ff (.Q(qq[i]), .C(timer), .CE(1'b1), .R(rst), .D(d[i]));
   enddo
endexpand

set q = qq;
```

۵-۲) زمان کل سیستم و progress time

همان‌طور که پیش‌تر توضیح داده شد، اجرای برنامه طبق یک مدل زمانی رخ می‌دهد. بدین منظور، می‌توان تصور کرد اجرای برنامه از لحظهٔ صفر آغاز شده و در بازه‌های مشخص زمانی، رویدادهای مشخصی اتفاق می‌افتند که چگونگی تناوب این رویدادها توسط دستوراتی مانند دستور repeat (قسمت 5-4) مشخص می‌شود و پیاده‌سازی این زمان‌بندی توسط کامپایلر صورت می‌پذیرد. به طور مثال اگر رویداد a و b، به ترتیب هر 5 و 8 ثانیه یک بار به طور متناوب تکرار شوند، پس از گذشت 5 ثانیه از پیشرفت زمان، رویداد a و بعد از 3 ثانیه رویداد b اتفاق افتاده و این تناوب به همین صورت ادامه خواهد یافت.

۵-۳) مفهوم timer و تیک‌های زمانی

مفهوم timer در زبان CompoScript، بیانگر تیک‌های متناوب زمانی است که در اکثر زبان‌های توصیف سیستم وجود دارد و گذشت رویدادها و زمان‌بندی و همگام‌سازی اجزای یک سیستم، توسط این متغیر مشخص می‌شود. در این زبان، یک کامپوننت می‌تواند متغیر timer را به عنوان آرگومان دریافت کرده و همگام‌سازی رویدادهای خود را با توجه به آن انجام دهد. این متغیر در کامپوننت مادر می‌تواند به صورت یک متغیر از جنس data تعریف شده و با استفاده از دستوراتی مانند repeat (قسمت 5-4)، به صورت متناوب مقدار آن عوض شود تا مفهوم تیک‌های زمانی پیاده‌سازی شود. یک مثال از این مورد در کد زیر قابل مشاهده است:

```
data timer0, timer1, rst;

Top_Counters dut(.timer0_in(timer0), .timer1_in(timer1), .rst(rst), .q0(q0), .q1(q1));

repeat (FOREVER) (5) timer0 = ~timer0;
```

۵-۴) تکرار تناوبی (repeat)

دستور repeat برای اجرای یک رویداد به صورت متناوب به کار می‌رود که قالب سینتکس آن به صورت می‌باشد. برای مثال، در قطعه کد زیر، دستور display(q0) به تعداد 10 مرتبه و هر 5 واحد زمانی و دستور display(q1) به تعداد 10 مرتبه و هر 8 واحد زمانی تکرار می‌شوند:

repeat (repeat_num) (interval) command

```
repeat (10) (5) display(q0)
repeat (10) (8) display(q1)
```

۶) تولید ساختاری

۶-۱) سازه‌ی do...enddo، index، expand

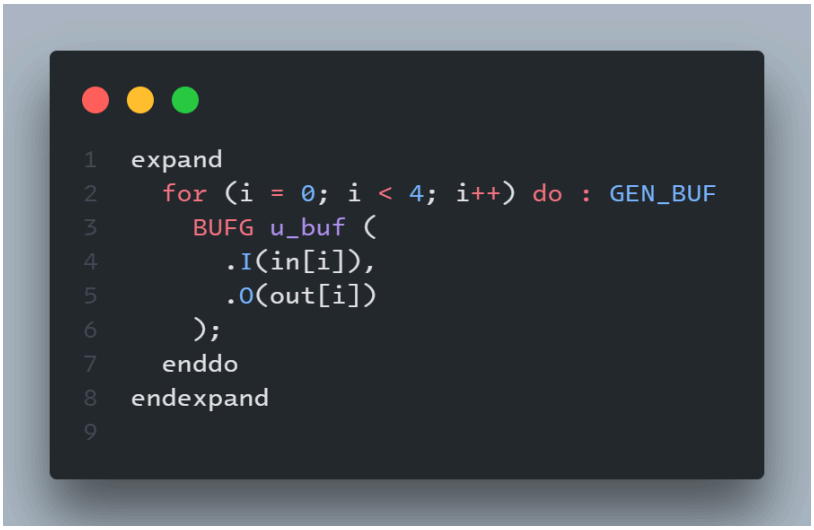
CompoScript برای تولید خودکار ساختارهای تکراری شامل سه بخش است:

do … enddo: برای تعریف یک بلوک ترتیبی از چند دستور استفاده می‌شود تا آن‌ها به‌عنوان یک واحد منطقی در ساختارهای کنترلی یا ترتیبی اجرا شوند.

index: شمارنده‌ی ایستای همان بلوکِ ترتیبی است که در هر گام مقدار عددی قطعی دارد.

expand: برای تولید ساختارهای تکراری در زمان ترجمه استفاده می‌شود و امکان ایجاد چند نمونه از ماژول‌ها، بلاک‌ها یا اتصالات را با استفاده از حلقه‌ها یا شرط‌ها فراهم می‌سازد.

مثال: قطعه‌کد زیر چهار نمونه از کامپوننت BUFG را به‌صورت خودکار ایجاد می‌کند.

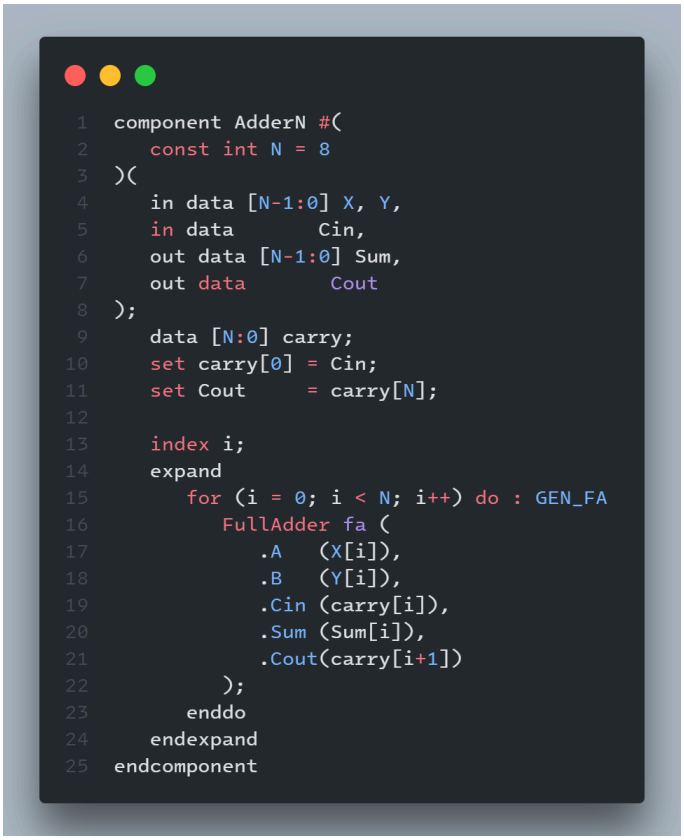


```
expand
  for (i = 0; i < 4; i++) do : GEN_BUF
    BUFG u_buf (
      .I(in[i]),
      .O(out[i])
    );
  enddo
endexpand
```

در این مثال، متغیر i در حلقه‌ی for از ۰ تا ۳ تغییر می‌کند. سازوکار expand باعث می‌شود در هر تکرار، یک نمونه‌ی جدید از کامپوننت BUFG با اتصالات in[i] و out[i] به‌صورت خودکار تولید شود؛ بنابراین لازم نیست چهار بار به‌صورت دستی نمونه‌های تکراری را بنویسیم.

۶-۲) الگوهای تکرار ساختاری و نام‌گذاری بلوک‌ها

از الگوهای تکرار ساختاری برای تولید چند نمونه از یک component یا بلاک استفاده می‌شود:



```
component AdderN #(
   const int N = 8
)(
   in data [N-1:0] X, Y,
   in data       Cin,
   out data [N-1:0] Sum,
   out data       Cout
);
   data [N:0] carry;
   set carry[0] = Cin;
   set Cout     = carry[N];

   index i;
   expand
      for (i = 0; i < N; i++) do : GEN_FA
         FullAdder fa (
            .A   (X[i]),
            .B   (Y[i]),
            .Cin (carry[i]),
            .Sum (Sum[i]),
            .Cout(carry[i+1])
         );
      enddo
   endexpand
endcomponent
```

به هر بلوک تکرار یک برچسب کوتاه و معنی‌دار اختصاص داده می‌شود که باید یکتا باشد. برای مثال در قطعه کد بالا، GEN_FA برچسب حلقه for است. پیاده‌سازی این قسمت می‌تواند در پیاده‌سازی فاز code generation به شما کمک کند.

۷) انواع داده

۷-۱) انواع داده‌های عددی (Numeric Data Types)

داده‌های عددی پایه‌ی اصلی بسیاری از پیکربندی‌ها را تشکیل می‌دهند.

در این زبان می‌توان عددها را به صورت ساده یا دارای اندازه‌ی مشخص تعریف کرد:

```
data counter = 0;
data [7:0] value = 8'd10;
data [3:0] delta = -2;
```

هر داده‌ی عددی می‌تواند در طول شبیه‌سازی تغییر کند (به‌صورت افزایشی، تفاضلی یا شرطی).

عملگرهایی مانند +, -, *, /, %, و عملگرهای منطقی &, |, ^, ~ برای آن‌ها قابل استفاده‌اند.

۲-۷) انواع داده‌های رشته‌ای (String Data Types)

رشته‌ها معمولاً برای پیام‌ها یا ترکیب داده‌های متنی در دستورات `display` به‌کار می‌روند.

به‌جای چاپ مستقیم چند مقدار، از ترکیب (concatenation) استفاده می‌شود:

```
display({"clk=", clk, ", data0=", data0, ", data1=", data1});
```

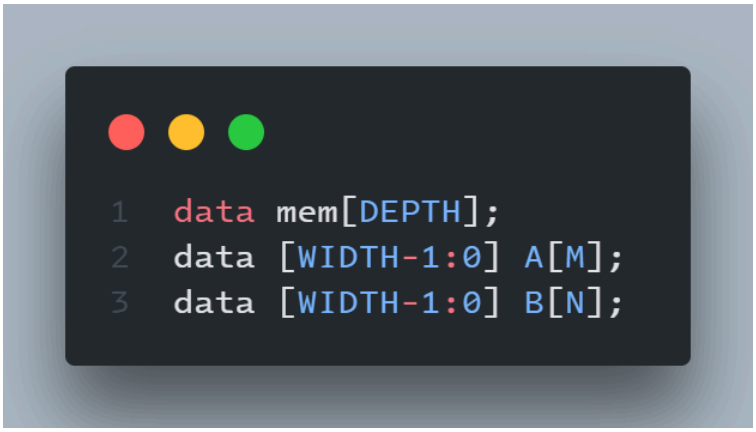
در این حالت، رشته‌ی درون آکولاد `{...}` شامل بخش‌های متنی و متغیرهاست که در زمان اجرا به‌صورت رشته‌ای یکی می‌شوند.

داده‌ی رشته‌ای ذاتاً نوع جداگانه‌ای ندارد و محصول ترکیب متغیرها و متن است؛ بنابراین انعطاف بالایی در ساخت پیغام‌های تشخیصی و خروجی‌ها فراهم می‌کند

۳-۷) انواع داده‌های آرایه‌ای (Array Types)

آرایه‌ها ساختاری برای مجموعه‌ای از داده‌ها با نوع یکسان هستند.

تعریف ساده‌ی آن‌ها به شکل زیر است:



آرایه‌ها می‌توانند تک‌بعدی یا چندبعدی باشند. در مثال بالا یک آرایه‌ی دو بعدی تعریف شده است.

استفاده از [lower:upper] برای تعیین محدوده استفاده از مقادیر یک آرایه می‌باشد.

دسترسی به عنصر خاص با اندیس انجام می‌شود. مانند A[i][j] یا mem[ptr]

۴-۷) داده‌های متنوع

این بخش به داده‌هایی اشاره دارد که ماهیت خاص (نه صرفا عدد یا آرایه) دارند:

- **پارامترها (parameter)** برای تنظیم اندازه‌ها، عمق‌ها و ضرایب به‌کار می‌روند:

```
parameter M = 2;
parameter WIDTH = 8;
parameter COEF1 = 2;
```

آن‌ها مقادیر ثابت در مقیاس پیکربندی هستند و معمولا قبل از اجرای شبیه‌سازی تعیین می‌شوند.

- **پورت‌ها (port in / port out)** واسطه‌ی داده بین بخش‌های مختلف پیکربندی‌اند:

```
port in  data [7:0] X,
port out data [15:0] Result
```

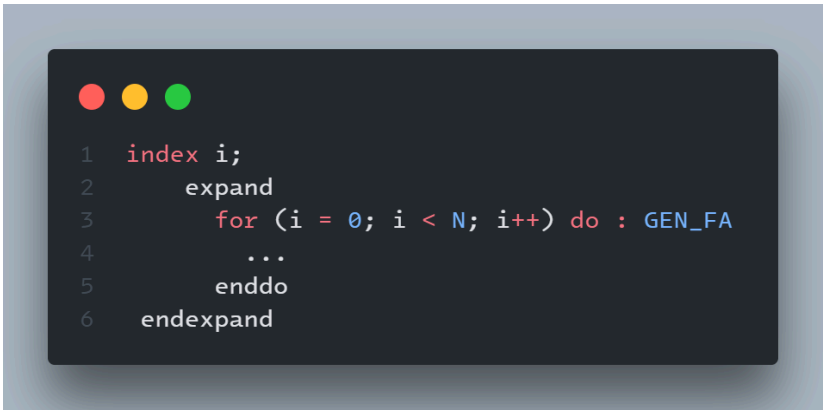
همین نوع داده‌ها در componenttype برای اتصال مولفه‌ها استفاده می‌شوند.

- **داده‌های شرطی**، که از عبارات سه‌تایی و شرط if تشکیل می‌شوند:

```
max = (a > b) ? a : b;
if (!rst && rd_en) dout = mem[rd_ptr];
```

این داده‌ها به‌صورت پویا در زمان اجرا مقدار می‌گیرند.

- **index** به‌عنوان شمارنده‌ی تولید:



متغیر i یک متغیر شمارنده موقت است که در زمان کامپایل ساختار استفاده می‌شود. هدف آن ایجاد اندیس‌های متوالی (۰، ۱، ۲، ...) برای نام‌گذاری یا اتصال زیرمولفه‌های تکراری است.

برخلاف متغیرهای data که در زمان اجرای شبیه‌سازی مقدارشان تغییر می‌کند، متغیر index در زمان ساخت مدل کار خود را انجام می‌دهد و در شبیه‌سازی نهایی نقشی ندارد.

- **timescale** یک دستور حیاتی در محیط‌های شبیه‌سازی است که واحد اندازه‌گیری زمان برای کل ماژول یا پیکربندی را تعیین می‌کند. ساختار آن به صورت زیر است:

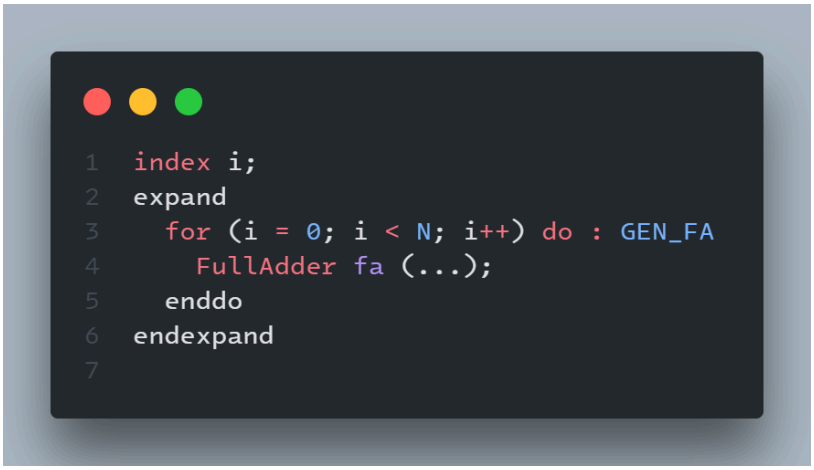
`timescale 1ns/1ps

این مقادیر شامل کوچکترین واحد زمانی است که برای شبیه‌سازی نهایی استفاده خواهند شد.

۵-۷) متغیرهای تولیدی (Generative Variables)

متغیرهای تولیدی بیشتر در تعریف اجزای تکرارشونده یا پارامتریک استفاده می‌شوند.

نمونه‌ی شاخص آن در مولفه AdderN است:



اینجا متغیر `i` فقط در زمان تولید ساختار (expansion) معتبر است و برای ایجاد چند نمونه از یک زیر-مولفه استفاده می‌شود.

این نوع متغیرها رفتار زمان‌اجرای شبیه‌سازی را تغییر نمی‌دهند؛ بلکه در مرحله‌ی «ساخت مدل» تعیین می‌کنند که چند نمونه‌ی تکراری ساخته شود.

در نتیجه، «متغیر تولیدی» ابزاری برای **پارامتریک‌سازی** و **تولید خودکار ساختارهای تکراری** است

۸) عملگرها

عملگرها در زبان CompoScript به چهار دسته ی عملگرهای حسابی، مقایسه ای، منطقی و تخصیص تقسیم میشوند.

۸-۱) عملگرهای حسابی

این دسته از عملگرها تنها روی اعداد عمل میکنند. لیست این عملگر ها در جدول زیر آمده است. در مثال های استفاده شده A را برابر با 20 و B را برابر با 10 درنظر بگیرید.

عملگر	شرکت‌پذیری	توضیح	مثال
+	چپ	جمع	A+B=30

-	چپ	تفریق	A–B=10
*	چپ	ضرب	A*B=200
/	چپ	تقسیم	A/ B = 2 B / A =0
mod	چپ	باقی‌مانده	A mod B=10
-	راست	منفی تک عمل‌وندی	-A = -20
-- و ++	راست	پیشوندی	--A
-- و ++	چپ	پسوندی	A++

۸-۲) عملگرهای مقایسه‌ای

این عملگرها وظیفه ی مقایسه را دارند؛ پس نتیجه ی آنها باید مقدار true یا false باشد. یعنی خروجی آنها یک boolean است.

توجه داشته باشید که عملوندهای عملگرهای > و < تنها از جنس اعداد صحیح هستند.

همچنین برای عملگر == و != نیز باید تایپ عملوندها یکسان باشند و در صورت آرایه بودن، اندازه ی آنها نیز برابر باشد. در غیر این صورت باید خطای کامپایل گرفته شود.

لیست عملگرهای مقایسه ای در جدول زیر آمده است. مقادیر A و B را مانند قبل در نظر بگیرید.

عملگر	شرکت‌پذیری	توضیح	مثال
==	چپ	تساوی	(A == B) = false

!=	چپ	عدم تساوی	(A != B) = true
<	چپ	کوچکتر	(A < B) = false
>	چپ	بزرگتر	(A > B) = true

۸-۳) عملگرهای منطقی

در این زبان، عملگرهای منطقی تنها روی تایپ Boolean قابل اعمال است. این عملگرها در جدول زیر آمده است. A را برابر true و B را برابر false در نظر بگیرید.

عملگر	شرکت‌پذیری	توضیح	مثال
&&	چپ	عطف منطقی	(A && B) = false
||	چپ	فصل منطقی	(A || B) = true
!	راست	نقیض منطقی	(!A) = false

۸-۵) عملگر تخصیص

این عملگر که به صورت = نمایش داده میشود وظیفه ی تخصیص را برعهده دارد. یعنی مقدار عملوند سمت راست را به عملوند سمت چپ اختصاص میدهد. برای آرایه مقدار تک تک عناصر آرایه ی سمت راست را به عناصر آرایه ی سمت چپ تخصیص میدهد. دقت شود که برای استفاده از این عملگر برای دو آرایه، اندازه ی آرایه ها باید برابر باشد.

همچنین دقت داشته باشید که عملوند سمت چپ باید از نوع left-value باشد. عبارات left-value عباراتی هستند که به یک مکان در حافظه اشاره میکنند. در مقابل عبارات right-value به مکان خاصی

در حافظه اشاره نمی‌کنند و صرفا یک عبارت دارای مقدار هستند. به عنوان مثال یک متغیر یا یک دسترسی به یکی از عناصر آرایه یک عبارت left-value است اما عبارت 3 + 10 یک عبارت right-value محسوب میشود. عبارات right-value تنها در سمت راست عملگر تخصیص قرار میگیرند.

۸-۶) اولویت عملگرها

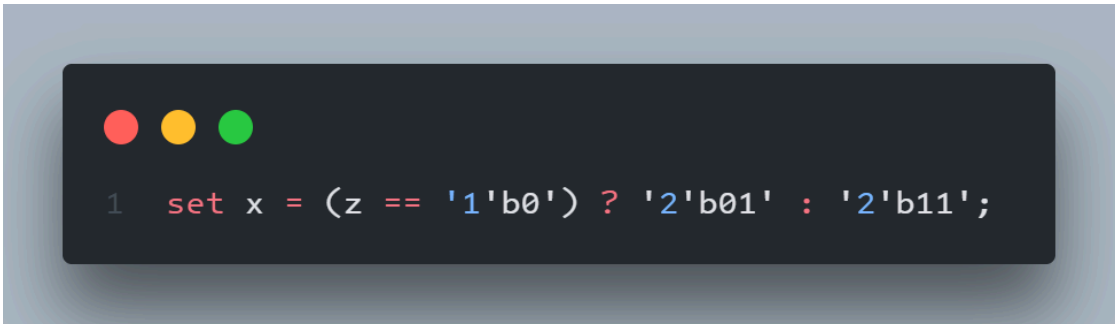
اولویت عملگرها طبق جدول زیر است:

اولویت	دسته	عملگرها	شرکت‌پذیری
1	پرانتز	()	چپ به راست
2	دسترسی به عناصر آرایه	[]	چپ به راست
3	تک عملوندی پسوندی	++ --	چپ به راست
4	تک عملوندی پیشوندی	- ! ++ --	راست به چپ
5	ضرب و تقسیم و باقی مانده	mod / *	چپ به راست
6	جمع و تفریق	+ -	چپ به راست
7	رابطه ای	< >	چپ به راست
8	مقایسه ی تساوی	=! ==	چپ به راست
9	عطف منطقی	&&	چپ به راست
10	فصل منطقی	||	چپ به راست
11	تخصیص	=	راست به چپ

12	کاما	,	چپ به راست

۷-۸) عملگر شرطی

در زبان CompoScript ساختار شرطی، condition ? true_statement : false_statement است.
مثالی از آن در زیر آمده است:



۹) توابع پیش‌فرض

در کد ها تابع پیش فرض display وجود دارد که برای نمایش متغیر های موجود استفاده می شود و به صورت زیر است:

